

AgentForge

A patch-generation and evaluation platform for the mini-swe-bench coding agent

Jacob Romer

May 6, 2026

Abstract

AgentForge is a patch-generation and evaluation platform for studying how tool design affects the performance of coding agents. Using mini-SWE-agent with Gemini 2.5 Flash, the project evaluates a suite of higher-level code-navigation, editing, and testing tools against a 40-instance SWE-bench Verified subset and a separate set of open GitHub issues. The refined tool suite increased the SWE-bench resolution rate from 41.7% to 56.8%, a 36% relative improvement, while intermediate regressions demonstrated that adding more capable tools can reduce performance when their interfaces introduce new failure modes.

Contents

Evaluation: SWE-bench verified-40	2
Experimental Setup	2
Performance	3
Tool Implementations	9
Learnings	19
Evaluation: Open GitHub Issues	21
Experimental Setup	21
Performance	22
Design	23
Learnings	31
Comparison with SWE-Bench instance runs	33

Evaluation: SWE-bench verified-40

Experimental Setup

The end-to-end pipeline runs `mini-extra swebench` against all 40 instances in the `verified-40` subset, writing per-instance trajectory files and a consolidated `preds.json` to subdirectories of the project root.

The SWE-bench harness (`swebench.harness.run_evaluation`) then evaluates each patch against the ground-truth test suite in a containerized environment using the same Docker image used for generation, producing `vertex_ai_gemini-2.5-flash.eval.json`.

- **Model:** `vertex_ai/gemini-2.5-flash`
- **Temperature:** `temperature=0.0`
- **Max steps:** `step_limit=250`
- **Token limits:** `cost_limit=$3.00` per instance
- **Number of instances evaluated:** 40
- **Pipeline command:** `scripts/run_verified40 final`
- **Trajectory inspection:** `scripts/view_trajectory`
- **Results reporting:** `scripts/extract_metrics <EVAL_JSON>`

Hardware

Each instance runs in an isolated `x86_64` container (`swebench/sweb.eval.x86_64.*:latest`) with a 60-second per-command timeout and a 2-hour container lifetime.

Artifact Locations

All artifacts live in subdirectories of `./<project-root>/<run-label>/`:

- Generation traces (compressed):
`./generation/<instance-id>/<instance-id>.traj.json.gz`
- Preds JSON: `./generation/preds.json`
- Evaluation JSON: `./vertex_ai_gemini-2.5-flash.eval.json`

Run Labels

- `./verified_40_baseline`: Baseline with only a basic `read_file` tool
- `./verified_40_run1`: Intermediate results, initial full tool set
- `./verified_40`: Single-run eval results, refined full tool set

Tool Implementations

	Tool Name	Purpose
1	<code>write_file (wf)</code>	Safer file edits
2	<code>read_file (rf)</code>	Targeted file reading
3	<code>search_in_file (sf)</code>	Grep within a single file
4	<code>find_definition (fd)</code>	Cross-file symbol search
5	<code>run_tests (rt)</code>	Test execution with discovery
6	<code>outline_file (of)</code>	File structure overview
7	<code>str_replace (sr)</code>	Safe exact-string replacement
8	<code>insert_lines (il)</code>	Insert content at specific line

Although mini-SWE-agent interacts with the environment through Bash commands, the tools implemented here provide higher-level primitives that the agent can invoke instead of composing fragile shell pipelines.

Performance

Final Results

Final-run per-instance status, showing 21 of 40 instances resolved, generated from `verified_40/vertex_ai__gemini-2.5-flash.eval.json`.

Instance ID	Status	Instance ID	Status
<code>django__django-10880</code>	Resolved	<code>django__django-16642</code>	Resolved
<code>django__django-11095</code>	Failed	<code>django__django-16661</code>	Failed
<code>django__django-11333</code>	Resolved	<code>psf__requests-2317</code>	Failed
<code>django__django-11532</code>	Resolved	<code>pydata__xarray-6461</code>	Resolved
<code>django__django-11880</code>	Resolved	<code>pydata__xarray-6992</code>	Failed
<code>django__django-13089</code>	Resolved	<code>pylint-dev__pylint-4661</code>	Failed
<code>django__django-13363</code>	Resolved	<code>pytest-dev__pytest-5809</code>	Resolved
<code>django__django-13417</code>	Failed	<code>pytest-dev__pytest-7982</code>	Resolved
<code>django__django-14007</code>	Failed	<code>scikit-learn__scikit-learn-13135</code>	Resolved
<code>django__django-14017</code>	Resolved	<code>scikit-learn__scikit-learn-13142</code>	Resolved
<code>django__django-14725</code>	Failed	<code>sphinx-doc__sphinx-8595</code>	Failed
<code>django__django-14855</code>	Resolved	<code>sphinx-doc__sphinx-8721</code>	Resolved
<code>django__django-15103</code>	Resolved	<code>sphinx-doc__sphinx-9229</code>	Failed
<code>django__django-15315</code>	Resolved	<code>sympy__sympy-13615</code>	Failed
<code>django__django-15368</code>	Failed	<code>sympy__sympy-13852</code>	Failed
<code>django__django-15554</code>	Failed	<code>sympy__sympy-14248</code>	Failed
<code>django__django-15863</code>	Resolved	<code>sympy__sympy-15017</code>	Resolved
<code>django__django-15987</code>	Resolved	<code>sympy__sympy-16792</code>	Resolved
<code>django__django-16136</code>	Failed	<code>sympy__sympy-17139</code>	Failed
<code>django__django-16560</code>	Failed	<code>sympy__sympy-19346</code>	Failed

Failed above aggregates all non-resolved outcomes (16 Unresolved, 2 Empty-Patch, 1 Error). Tables 1–4 below break these out individually.

Summary

Metric	Baseline	Intermediate	Final
Submitted instances	40	40	40
Completed instances	36	35	37
Empty-Patch instances (–)	3	2	2
Error instances (△)	1	3	1
Resolved instances (✓)	15	19	21
Unresolved instances (×)	21	16	16
Resolution rate (Resolved / Completed)	41.7%	54.3%	56.8%

The agent successfully resolved 15 of the 36 completed instances in the baseline evaluation, and 21 of the 37 completed instances with tool implementations, corresponding to a 36% improvement in the resolution rate.

Interestingly, our intermediate results demonstrated degraded performance relative to the baseline milestone, which used only two tool implementations (and, critically, a simpler implementation of the `write_file` tool), illustrating that more (or more capable) tools are not necessarily better, if they are overly complex or counterintuitively designed.

Subsequent simplification of the `write_file` tool interface and extraction of functionality into dedicated `str_replace` and `insert_lines` tools restored the previous performance.

Results

Detailed per-instance results are outlined in Tables 1, 2, and 3 below. Along with the instance status: Resolved ✓, Failed ×, Empty –, or Error △, the number of steps, patch size, and tool call metrics are collected.

Table 4 compares the per-instance baseline to final results (instance status, step count, and patch size).

Individual trajectory files can be inspected from the command line using `scripts/view_trajectory`.

#	Instance ID	Status	Steps	Patch Size	Tool Calls	Tools Breakdown
1	django__django-10880	✓	28	39	4	read_file(4)
2	django__django-11095	✓	44	36	2	read_file(2)
3	django__django-11333	×	48	10	3	read_file(3)
4	django__django-11532	✓	22	47	2	read_file(2)
5	django__django-11880	✓	14	3	1	read_file(1)
6	django__django-13089	✓	6	11	2	read_file(2)
7	django__django-13363	✓	12	6	2	read_file(2)
8	django__django-13417	✓	43	119	5	read_file(5)
9	django__django-14007	×	106	111	0	-
10	django__django-14017	×	25	11	1	read_file(1)
11	django__django-14725	×	19	20	2	read_file(2)
12	django__django-14855	✓	14	4	1	read_file(1)
13	django__django-15103	×	11	20	4	read_file(4)
14	django__django-15315	✓	9	30	2	read_file(2)
15	django__django-15368	×	32	133	1	read_file(1)
16	django__django-15554	×	33	80	3	read_file(3)
17	django__django-15863	×	35	56	2	read_file(2)
18	django__django-15987	×	8	5	1	read_file(1)
19	django__django-16136	×	36	178	13	read_file(13)
20	django__django-16560	×	13	14	1	read_file(1)
21	django__django-16642	✓	17	4	1	read_file(1)
22	django__django-16661	×	33	2147	5	read_file(5)
23	psf__requests-2317	×	6	4	1	read_file(1)
24	pydata__xarray-6461	✓	59	69	3	read_file(3)
25	pydata__xarray-6992	×	7	11	1	read_file(1)
26	pylint-dev__pylint-4661	×	8	9	3	read_file(3)
27	pytest-dev__pytest-5809	✓	9	4	4	read_file(4)
28	pytest-dev__pytest-7982	✓	8	4	1	read_file(1)
29	scikit-learn__scikit-learn-13135	✓	34	14	3	read_file(3)
30	scikit-learn__scikit-learn-13142	-	250	-	3	read_file(3)
31	sphinx-doc__sphinx-8595	×	18	4	3	read_file(3)
32	sphinx-doc__sphinx-8721	×	7	4	2	read_file(2)
33	sphinx-doc__sphinx-9229	△	12	0	2	read_file(2)
34	sympy__sympy-13615	-	250	-	1	read_file(1)
35	sympy__sympy-13852	-	250	-	6	read_file(6)
36	sympy__sympy-14248	×	89	70	4	read_file(4)
37	sympy__sympy-15017	✓	50	21	10	read_file(10)
38	sympy__sympy-16792	×	45	43	5	read_file(5)
39	sympy__sympy-17139	×	10	17	1	read_file(1)
40	sympy__sympy-19346	×	30	180	6	read_file(6)

Table 1: Baseline results

#	Instance ID	Status	Steps	Patch Size	Tool Calls	Tools Breakdown
1	django__django-10880	×	56	1524	26	rf(8), rt(2), sf(3), sr(1), wf(12)
2	django__django-11095	✓	15	8	7	fd(1), of(1), rf(5)
3	django__django-11333	✓	31	2191	15	of(1), rf(6), rt(2), sf(1), sr(3), wf(2)
4	django__django-11532	✓	32	34	12	rf(6), rt(2), wf(4)
5	django__django-11880	✓	40	1253	19	rf(12), rt(2), wf(5)
6	django__django-13089	×	20	301	15	rf(10), sf(1), sr(1), wf(3)
7	django__django-13363	✓	33	166	12	of(1), rf(8), rt(2), sr(1)
8	django__django-13417	✓	35	1988	9	fd(4), rf(2), sf(2), wf(1)
9	django__django-14007	✓	70	2258	46	rf(20), sf(22), sr(4)
10	django__django-14017	×	57	434	25	of(1), rf(16), rt(1), sf(2), sr(1), wf(4)
11	django__django-14725	–	250	–	114	of(2), rf(88), rt(15), sr(4), wf(5)
12	django__django-14855	✓	6	451	4	rf(2), sf(1), sr(1)
13	django__django-15103	×	23	1623	16	rf(6), rt(3), sf(2), sr(3), wf(2)
14	django__django-15315	✓	25	2550	7	fd(4), rf(1), sf(1), sr(1)
15	django__django-15368	✓	24	3265	11	rf(4), rt(4), sr(1), wf(2)
16	django__django-15554	△	51	0	43	rf(19), rt(11), sf(3), wf(10)
17	django__django-15863	×	24	1026	13	rf(6), sf(3), sr(2), wf(2)
18	django__django-15987	✓	8	4	4	of(1), rf(3)
19	django__django-16136	✓	27	371	11	fd(3), rf(4), sf(1), wf(3)
20	django__django-16560	△	52	0	16	of(1), rf(9), sf(2), wf(4)
21	django__django-16642	×	23	789	6	of(1), rf(2), sr(3)
22	django__django-16661	×	14	2522	9	rf(5), rt(2), sf(1), wf(1)
23	psf__requests-2317	×	31	97	11	of(1), rf(6), sf(4)
24	pydata__xarray-6461	✓	12	2017	6	rf(3), sf(2), wf(1)
25	pydata__xarray-6992	×	6	8949	2	rf(1), wf(1)
26	pylint-dev__pylint-4661	×	33	130	19	rf(9), sf(2), sr(7), wf(1)
27	pytest-dev__pytest-5809	✓	7	120	5	rf(3), sr(2)
28	pytest-dev__pytest-7982	✓	7	4	3	rf(1), sf(2)
29	scikit-learn__scikit-learn-13135	✓	16	312	8	of(1), rf(3), wf(4)
30	scikit-learn__scikit-learn-13142	✓	39	1170	16	fd(1), of(2), rf(10), wf(3)
31	sphinx-doc__sphinx-8595	△	65	0	28	of(2), rf(16), sf(1), sr(1), wf(8)
32	sphinx-doc__sphinx-8721	×	5	302	2	rf(1), wf(1)
33	sphinx-doc__sphinx-9229	×	73	601	36	of(4), rf(22), sf(1), sr(2), wf(7)
34	sympy__sympy-13615	×	49	2287	29	fd(2), rf(13), sf(6), sr(8)
35	sympy__sympy-13852	×	12	584	4	of(1), rf(2), sr(1)
36	sympy__sympy-14248	–	250	–	104	fd(1), rf(36), sf(9), sr(36), wf(22)
37	sympy__sympy-15017	×	68	4	21	fd(3), of(2), rf(10), sf(5), sr(1)
38	sympy__sympy-16792	✓	37	2281	24	of(2), rf(13), sf(7), sr(1), wf(1)
39	sympy__sympy-17139	✓	35	4451	22	rf(10), rt(6), sr(2), wf(4)
40	sympy__sympy-19346	×	23	18	3	rf(3)

Table 2: Intermediate results

#	Instance ID	Status	Steps	Patch Size	Tool Calls	Tools Breakdown
1	django__django-10880	✓	32	22	19	il(3), of(1), rf(4), rt(6), sr(5)
2	django__django-11095	×	16	8	13	fd(5), il(2), of(1), rf(3), sf(1), sr(1)
3	django__django-11333	✓	23	16	11	fd(1), il(1), rf(2), rt(1), sf(3), sr(3)
4	django__django-11532	✓	18	9	7	il(1), rf(2), rt(2), sf(1), sr(1)
5	django__django-11880	✓	57	29	26	il(9), of(1), rf(12), rt(4)
6	django__django-13089	✓	28	10	17	il(1), of(1), rf(9), rt(2), sr(4)
7	django__django-13363	✓	26	6	6	rf(2), rt(2), sf(1), sr(1)
8	django__django-13417	×	98	24	33	fd(3), il(9), rf(15), sf(4), sr(2)
9	django__django-14007	×	28	30	22	il(1), of(3), rf(11), sr(7)
10	django__django-14017	✓	35	48	21	fd(1), il(5), of(2), rf(7), sr(4), wf(2)
11	django__django-14725	×	78	1429	52	il(9), of(1), rf(31), rt(7), sr(4)
12	django__django-14855	✓	9	8	6	fd(1), rf(1), sf(3), sr(1)
13	django__django-15103	✓	29	25	20	fd(1), il(1), rf(3), rt(7), sf(2), sr(6)
14	django__django-15315	✓	22	48	6	fd(3), rf(1), sf(1), sr(1)
15	django__django-15368	×	23	4	12	fd(1), il(1), of(1), rf(3), rt(3), sf(1), sr(2)
16	django__django-15554	×	30	59	14	fd(2), of(2), rf(9), sr(1)
17	django__django-15863	✓	14	35	8	il(1), of(1), rf(4), sf(2)
18	django__django-15987	✓	5	4	2	rf(1), sr(1)
19	django__django-16136	×	86	54	51	fd(1), il(6), of(2), rf(20), rt(15), sf(3), sr(3), wf(1)
20	django__django-16560	×	31	32	20	fd(2), il(4), of(1), rf(10), sr(3)
21	django__django-16642	✓	12	37	5	fd(2), il(1), of(1), rf(1)
22	django__django-16661	×	55	44	20	fd(2), il(1), rf(5), rt(5), sf(4), sr(3)
23	psf__requests-2317	×	13	41	8	rf(3), rt(2), sf(2), sr(1)
24	pydata__xarray-6461	✓	83	39	54	fd(2), il(5), of(1), rf(26), sf(7), sr(13)
25	pydata__xarray-6992	×	14	11	7	il(1), rf(5), sr(1)
26	pylint-dev__pylint-4661	×	9	6	6	fd(2), rf(2), sr(2)
27	pytest-dev__pytest-5809	✓	5	4	3	rf(1), sf(1), sr(1)
28	pytest-dev__pytest-7982	✓	6	4	4	rf(1), sf(2), sr(1)
29	scikit-learn__scikit-learn-13135	✓	29	3	6	rf(6)
30	scikit-learn__scikit-learn-13142	✓	17	24	8	fd(2), il(1), of(2), rf(3)
31	sphinx-doc__sphinx-8595	△	20	22802	3	rf(1), sf(1), sr(1)
32	sphinx-doc__sphinx-8721	✓	37	4	22	il(7), rf(14), sf(1)
33	sphinx-doc__sphinx-9229	×	20	18	16	fd(6), il(2), rf(3), sf(5)
34	sympy__sympy-13615	–	250	–	130	fd(6), il(108), rf(8), sf(8)
35	sympy__sympy-13852	×	11	39	6	fd(2), of(1), rf(2), sr(1)
36	sympy__sympy-14248	–	250	–	80	fd(3), il(52), of(1), rf(17), sr(7)
37	sympy__sympy-15017	✓	21	11	11	of(2), rf(6), sf(2), sr(1)
38	sympy__sympy-16792	✓	113	44	50	fd(6), il(8), rf(22), sr(14)
39	sympy__sympy-17139	×	8	6	3	rf(1), sr(2)
40	sympy__sympy-19346	×	49	13	15	fd(1), il(2), of(1), rf(10), sf(1)

Table 3: Final results

#	Instance ID	Status	Steps	Patch Size
1	django__django-10880	✓ → ✓	28 → 32	39 → 22
2	django__django-11095	✓ → ×	44 → 16	36 → 8
3	django__django-11333	× → ✓	48 → 23	10 → 16
4	django__django-11532	✓ → ✓	22 → 18	47 → 9
5	django__django-11880	✓ → ✓	14 → 57	3 → 29
6	django__django-13089	✓ → ✓	6 → 28	11 → 10
7	django__django-13363	✓ → ✓	12 → 26	6 → 6
8	django__django-13417	✓ → ×	43 → 98	119 → 24
9	django__django-14007	× → ×	106 → 28	111 → 30
10	django__django-14017	× → ✓	25 → 35	11 → 48
11	django__django-14725	× → ×	19 → 78	20 → 1429
12	django__django-14855	✓ → ✓	14 → 9	4 → 8
13	django__django-15103	× → ✓	11 → 29	20 → 25
14	django__django-15315	✓ → ✓	9 → 22	30 → 48
15	django__django-15368	× → ×	32 → 23	133 → 4
16	django__django-15554	× → ×	33 → 30	80 → 59
17	django__django-15863	× → ✓	35 → 14	56 → 35
18	django__django-15987	× → ✓	8 → 5	5 → 4
19	django__django-16136	× → ×	36 → 86	178 → 54
20	django__django-16560	× → ×	13 → 31	14 → 32
21	django__django-16642	✓ → ✓	17 → 12	4 → 37
22	django__django-16661	× → ×	33 → 55	2147 → 44
23	psf__requests-2317	× → ×	6 → 13	4 → 41
24	pydata__xarray-6461	✓ → ✓	59 → 83	69 → 39
25	pydata__xarray-6992	× → ×	7 → 14	11 → 11
26	pylint-dev__pylint-4661	× → ×	8 → 9	9 → 6
27	pytest-dev__pytest-5809	✓ → ✓	9 → 5	4 → 4
28	pytest-dev__pytest-7982	✓ → ✓	8 → 6	4 → 4
29	scikit-learn__scikit-learn-13135	✓ → ✓	34 → 29	14 → 3
30	scikit-learn__scikit-learn-13142	→ → ✓	250 → 17	→ → 24
31	sphinx-doc__sphinx-8595	× → △	18 → 20	4 → 22802
32	sphinx-doc__sphinx-8721	× → ✓	7 → 37	4 → 4
33	sphinx-doc__sphinx-9229	△ → ×	12 → 20	0 → 18
34	sympy__sympy-13615	→ → -	250 → 250	→ → -
35	sympy__sympy-13852	→ → ×	250 → 11	→ → 39
36	sympy__sympy-14248	× → -	89 → 250	70 → -
37	sympy__sympy-15017	✓ → ✓	50 → 21	21 → 11
38	sympy__sympy-16792	× → ✓	45 → 113	43 → 44
39	sympy__sympy-17139	× → ×	10 → 8	17 → 6
40	sympy__sympy-19346	× → ×	30 → 49	180 → 13

Table 4: Baseline results compared to final

Tool Implementations

write_file: Safer file edits

Motivation The system prompt suggests using bash heredocs (`cat <<'EOF' > file`) and `sed -i` for file edits, but both approaches are fragile in a subshell-per-command environment:

- **Shell escaping:** heredoc content containing single quotes, backslashes, or the EOF delimiter itself corrupts the write silently or raises a syntax error.
- **sed precision:** replacing a known string with `sed` fails when the target string is dynamic, spans multiple lines, or contains regex metacharacters. Agents frequently made incorrect replacements or had to issue several corrective `sed` calls.
- **Lack of atomicity:** a failed partial write leaves the file in an inconsistent state with no rollback.

A dedicated `write_file` tool eliminates these failure modes by bypassing the shell quoting and performing file writes through a controlled interface.

Interface

```
write_file(  
    path: str, # destination file path; parent dirs created automatically  
    content: str | None = None, # agent supplies entire file content  
    content_b64: str | None = None, # alternative for hard-to-escape content  
    mode: "overwrite" | "append" = "overwrite",  
)
```

The plain and base64 content variants are mutually exclusive.

Returns a non-zero returncode with an error message in case of failure. On success, returns a dictionary of the form

```
{"output": "Wrote N lines to <path> (overwrite).", "returncode": 0}
```

Implementation In the Docker environment, passing arbitrary content directly to the shell risks hitting `ARG_MAX` and shell-interpretation issues. The implementation avoids both by base64-encoding the content in Python, then decoding it inside the container:

```
base64 -d > /path/to/file <<'_CONTENT_'  
<base64-encoded content>  
_CONTENT_
```

The `_CONTENT_` delimiter is safe because the base64 alphabet never contains underscores. Parent directories are created first with `mkdir -p $(dirname <path>)`. In the local (non-Docker) environment, the implementation falls back to `Path.write_text()` / `open('a')` directly.

The main tradeoff is that overwrite mode rewrites the entire file on every call. For large files (hundreds of lines) this is token-expensive since the agent must supply the full content.

Results The `write_file` tool was invoked sparingly in the final run (three times across two of the 40 instances) because `str_replace` and `insert_lines` displaced almost every targeted-edit use case. The remaining calls illustrate the two roles whole-file writes still served: creating new files from scratch and making broad rewrites.

In `django__django-14017` (Resolved, 35 steps, 2 `write_file` calls), the agent used `write_file` for file creation rather than a targeted source edit. This is a use case `str_replace` cannot serve cleanly, since there is no existing string to replace, and one where `insert_lines` would require pre-creating an empty file. `write_file`'s overwrite semantics fit exactly.

In `django__django-16136` (Unresolved, 86 steps, 1 `write_file` call), the agent took the opposite approach: rewrote the entirety of `tests/async/tests.py` via `content_b64` to fix what it believed was an indentation problem. The call executed without error, but the agent's diagnosis of the underlying bug was wrong, and the rewrite did not resolve the failing test. The instance illustrates the trade-off flagged in the Implementation paragraph: whole-file rewrite is token-expensive and lets a confused agent commit a confidently-wrong large change in one shot, where `str_replace`'s uniqueness check would have at minimum forced a localized hypothesis.

read_file: Targeted file reading

Motivation The default reader approach of cat-ing entire files is token-inefficient for large codebases. A file like `django/db/models/query.py` is over 2,000 lines. Dumping it in full wastes most of the context window on irrelevant code and risks triggering the 10,000-character output truncation warning, which strips the middle of the file and forces follow-up reads. Targeted reading with `start_line/end_line` or `head/tail` lets the agent zero in on the specific region identified in the issue description, reducing token waste and keeping the context focused for accurate patching.

Interface

```
read_file(  
    path: str,  
    start_line: int | None = None,    # 1-indexed, inclusive  
    end_line: int | None = None,      # 1-indexed, inclusive  
    head: int | None = None,          # return first N lines  
    tail: int | None = None,          # return last N lines  
    with_line_numbers: bool = False, # prefix with line numbers  
    max_lines: int = 2000,            # hard truncation cap  
)
```

Only one slicing mode (`start_line/end_line`, `head`, or `tail`) may be used per call. Returns `{"output": str, "returncode": int}`. Output includes a truncation notice if lines were cut.

Implementation In the Docker environment, each slicing mode maps to a shell primitive executed via `env.execute()`: `head -n N`, `tail -n N`, or `sed -n 'S,Ep'`. This keeps implementation simple and avoids reading file contents into Python memory inside the container. In the local environment, the file is read with `Path.read_text()` and sliced in Python.

The most notable challenge is computing correct line numbers when using `tail` with `with_line_numbers=True`: the line offset is not known until after `wc -l` is called on the file, requiring a second shell round-trip before the `tail` read.

Output is hard-capped at `max_lines` (default 2,000) regardless of slicing mode, with a trailing `... (truncated N additional lines)` notice so the agent knows the output was cut.

Results The current `read_file` is a refined version of the simpler tool used in the baseline run. The slicing parameters (`start_line / end_line`, `head / tail`, `with_line_numbers`) and the `max_lines` truncation guard described above were added in response to the failure modes observed there. The two halves of the writeup correspond.

The motivating failure is visible in `django__django-16136` in the baseline (Unresolved, 36 steps, 13 `read_file` calls against the simpler tool). Across the full baseline run, a number of failed instances exhibit a variant of this “circular reads” anti-pattern: when the agent does not know where to look, an unbounded full-file read invites it to re-fetch content as a stand-in for actual progress. The slicing parameters and the `max_lines` cap were introduced specifically to make targeted reads cheap and full-file dumps loud (truncation notice on the way out).

The refinement does its job in the final run when the agent has a line target. `scikit-learn__scikit-learn-13142` (Resolved, 17 steps, 3 `read_file` calls) shows the pattern: after `outline_file` surfaced `fit_predict` at line 194 and `predict` at line 358 of `sklearn/mixture/base.py`, the agent issued three slice reads, each with `with_line_numbers=True`. No full-file read of `base.py` was ever issued. The fix landed in 17 steps versus the 250-step empty-patch timeout in the baseline.

The refinement is not, however, a complete defense against confusion. `django__django-14725` (Final: Unresolved, 78 steps, 31 `read_file` calls) shows the same “doesn’t know where to look” pattern reappearing even with the slicing tools available: The agent issued many reads across many files but never localized the fix. High `read_file` counts remain a confusion signal more than a thoroughness one. The targeted-reading interface helps when the agent has a target. It does not supply the target itself, which is where `outline_file`, `search_in_file`, and `find_definition` take over.

search_in_file: grep within a single file

Motivation The dominant failure pattern in the baseline was circular reads: the agent reading the same large file 10–15 times across a single trajectory while trying to locate a small region of interest. The agent could in principle invoke `grep` from `bash`, but in practice it composed `grep | head` pipelines inconsistently and frequently mis-escaped regex metacharacters in its shell quoting. A first-class single-file regex tool with a stable interface and bounded output eliminates the failure mode at its source.

Interface

```
search_in_file(  
    path:      str,  
    pattern:   str,          # Python regex (re.search semantics)  
    context:   int = 0,      # # of lines before/after each match  
    max_lines: int = 2000,   # hard truncation cap  
)
```

Match lines are prefixed with `<lineno>:`, context lines with `<lineno>-`, disjoint context windows are separated by `--`. Matches `grep`’s default output shape closely enough that the agent’s prior intuitions transfer.

Implementation The implementation reads the file (Docker: `cat --<quoted-path>` via `env.execute()`, local: `Path.read_text(errors="replace")`), compiles the supplied pattern with Python’s `re` module, and walks the line list to build a match set. Each match index is then expanded by context lines in either direction. Overlapping windows are merged via set union and emitted in sorted order. Output is hard-capped at `max_lines` with a trailing `...` (truncated `N` additional lines) notice. Errors are mapped to distinct return codes so the agent can disambiguate failures without re-parsing message text.

The notable tradeoff is the choice of Python `re` rather than POSIX/PCRE semantics. This avoids forking a subprocess and keeps escaping consistent across operating systems, but it does mean the agent occasionally writes a regex that would work in shell `grep` but raises in Python (e.g., `<word\>` word boundaries, or PCRE-only constructs). The strict-failure path (returncode 2 with the `re.error` message) is preferable to silent misses, which would be the alternative.

Results `search_in_file` was used frequently in the final run, tied with `run_tests` at 56 calls. The clearest table-supported illustration is `pytest-dev__pytest-5809` (Resolved, 5 steps, 1 `search_in_file` call). The baseline trajectory for the same instance solved it as well, but took nine steps and relied only on `read_file`. This is the kind of narrow localization problem the tool was intended to compress.

In `pydata__xarray-6461` (Resolved, 83 steps, 7 `search_in_file` calls), the tool did double duty: the agent used it to find candidate edit sites and then to re-verify

line numbers after each `str_replace`, since intervening edits had shifted offsets. This “search-edit-search” pattern shows up across the longer-trajectory successes and explains why a high `search_in_file` count is not, on its own, a signal of confusion the way a high `read_file` count is.

find_definition: Cross-file symbol search

Motivation Several baseline failures involved cross-file localization. A natural alternative is `grep -rn`, which works but produces a flood of unrelated call sites and import lines for any common name (`Field`, `Exists`). A definition-anchored search function promised to cut localization steps sharply for the symbol-lookup pattern.

Interface

```
find_definition(  
    symbol:    str,  
    path:      str = "/testbed", # required prefix in sandbox  
    docs:      bool = False, # search docstrings/comments instead  
    max_lines: int = 2000,  
)
```

In symbol mode, output is a list of `<file>: <lineno>` blocks with up to two context lines per match. In docs mode, the symbol is treated as a free-text search term against #-comments and triple-quoted docstrings.

Implementation The symbol path constructs four definition-anchoring regexes:

- `def`
- `async def`
- `class`
- top-level and class-level (indented) assignment

and runs each against the target directory. Ripgrep is preferred (`rg --line-number --context 2 --glob '*.py'`), and a `grep -rn -E --include=*.py` invocation is the fallback. Results from all four regexes are deduplicated by `(file, lineno)` and formatted via `format_match()`. In docs mode, a single heuristic regex targets `#/"'/'-bracketed` lines case-insensitively, with a broader unanchored fallback if the heuristic returns nothing. Sandbox usage is restricted to paths under `/testbed` to prevent unintended access, and non-conforming paths return `returncode 2` with an explanatory message.

The intended tradeoff was definition-only precision over `grep`’s recall: a clean list of fixpoints at the cost of missing dynamic definitions (metaclass-generated methods, `setattr`-injected attributes, factory functions). In practice a different tradeoff dominated, described next.

Results The tool was invoked across 22 of the 40 final-run instances but never returned a non-empty result on any call. Across cases like `django__django-11333`

(get_resolver), django__django-14855 (get_admin_url), django__django-13417 (QuerySet.ordered, default_ordering), sympy__sympy-16792 (make_routine, CodeGen, autowrap), and scikit-learn__scikit-learn-13142 (GaussianMixture), every call returned No definitions found for '<symbol>'.

The agent recognized the pattern quickly and consistently fell back to shell search, so the tool's effective contribution to resolution rate was approximately zero.

The suspected root cause is a host/container split in the dispatch logic: `has_rg()` calls `shutil.which("rg")` in the local Python process, then issues an `rg-syntax` command via `env.execute()` to the Docker container. If the container does not have `rg` installed (the SWE-bench x86_64 images are minimal), the command fails silently, the parser sees empty output, and the tool reports “no definitions found.” The fallback to `grep` should have been gated on a container-side check rather than a host-side one. A secondary UX problem is that the agent often supplied a relative path on first invocation and got the path-constraint error (`path must be under /testbed`), spending one or two steps recovering before the tool would even attempt the search.

Together these issues turned what was projected to be the second-highest-impact tool into dead weight. This was an implementation failure that should have been caught through early delivery and testing against real `verified-40` instances rather than exclusive reliance on unit tests.

run_tests: Test execution with discovery

Motivation None of the failed baseline trajectories had access to a first-class validation tool. The baseline configuration could still succeed on simple patches, but failed cases often spent their extra steps wandering rather than validating. A first-class test runner, invokable as a tool rather than a fragile bash composition, was meant to close that validation gap and, secondarily, to filter `pytest`'s verbose output.

Interface

```
run_tests(  
    path: str = "", # test file/dir (empty = repo root)  
    timeout: int = 60, # seconds before pytest is killed  
)
```

Returns the conventional `{"output": str, "returncode": int}` shape. Filtering of `pytest` output was deferred to `pytest`'s own `--tb=short -q` flags rather than implemented as custom parsing.

Implementation The tool composes `python -m pytest <path?> --tb=short -q`, prepends `cd /testbed && timeout <N>` for sandboxed runs, and dispatches via `env.execute()` (Docker) or `subprocess.run(..., shell=True, timeout=...)` (local). The path argument is `shlex.quoted`. Stdout and stderr

are merged into a single output string for local runs, and the Docker path returns whatever `env.execute()` produces.

The simplifying choices to hardcode pytest and defer summarization to pytest's own `-q` mode kept the implementation small, but produced two systemic gaps that dominated the Results below. The original `tools.md` design called for structured `{"passed": N, "failed": M, "failures": [...]}` output that strips boilerplate and collapses passing-test dots. That aspect was not built.

Results The tool was invoked across 12 of the 40 final-run instances, but its useful contribution was secondary rather than direct. Two limitations explain why.

Django incompatibility. Django's test suite is `unittest`-based and orchestrated through `tests/runtests.py`, not `pytest`. Django invocations frequently followed the same pattern: first `pytest` was unavailable, and later `pytest`-based runs failed with `ImproperlyConfigured: Requested setting INSTALLED_APPS, but settings are not configured.` Across `django__django-13089`, `django__django-13363`, `django__django-15103`, `django__django-11532`, and others, no `run_tests` invocation produced a clean Django pass/fail signal. The agent eventually had to pivot to `bash (python tests/runtests.py <app>)`.

Hardcoded container timeout. The 60-second per-command timeout enforced by the SWE-bench Docker harness overrode the tool's own `timeout` argument. In trajectories where the agent raised the tool timeout above 60 seconds, the observed failure still came from the harness-level limit.

The redeeming finding is that the tool's presence still influenced behavior even when the tool itself failed. `django__django-10880` was resolved in the final run after 6 `run_tests` attempts pushed the agent toward Django's own test runner, producing a 22-character patch compared to the baseline's 39-character patch. The implementation needs a runner-discovery step (try `pytest`, fall back to `manage.py test / runtests.py`) and either a configurable container timeout or a warning that values above 60 are silently clamped.

outline_file: File structure overview

Motivation Large Python files in the benchmark often have 1,000-2,000 lines, and content-oriented tools alone do not tell the agent how those files are organized. `read_file` and `search_in_file` both return lines from inside the file; neither answers "what is in this file?" without dumping its body. A structural overview displaying class and function signatures with line numbers is the missing primitive.

Interface

```
outline_file(  
    path:          str,  
    include_docstrings: bool = False, # Python only: first line  
    max_lines:     int  = 2000,  
)
```

Output is a flat list of `<lineno>: <signature>` entries with indentation reflecting nesting (e.g., a method indents one level under its class).

Implementation For Python files (.py suffix), the implementation parses the file with `ast.parse()` and walks the tree via `_ast_outline()`, emitting full signatures recovered via `ast.unparse()`: parameter lists, return-type annotations, base classes, keyword arguments to class, and decorator lines. On `SyntaxError` (or for non-Python suffixes), it falls back to a regex pattern that recognizes function/class/interface definitions across Go, TypeScript, Rust, and similar languages. In Docker mode, the file is fetched via `cat -- <quoted-path>` after a test `-d` probe distinguishes directories from missing files; locally, `Path.read_text(errors="replace")` is used.

The notable tradeoff is precision-vs-coverage: AST parsing produces exact, decorator-aware signatures but only for syntactically valid Python; the regex fallback is language-agnostic but conflates strings or comments that happen to match the pattern. In practice the AST path covered every Python file encountered in the verified-40 run.

Results The clearest demonstration is `scikit-learn__scikit-learn-13142` (Resolved, 17 steps, 2 `outline_file` calls), which had timed out at 250 steps with an empty patch in the baseline. The agent located `gaussian_mixture.py`, called `outline_file` once, and immediately saw that `GaussianMixture` (line 434) extended `BaseMixture` while the methods named in the issue (`fit_predict`, `predict`) were not defined locally. A second `outline_file` call on `base.py` returned `fit_predict` at line 194 and `predict` at line 358, after which the agent went directly to those line ranges and produced the patch. The two outlines together, with under 50 lines of output, replaced what would otherwise have been multiple thousand-line file reads.

The same pattern appears in `sympy__sympy-15017` (Resolved, 21 steps, 2 `outline_file` calls). The tool's behavior was uniform across calls, making it the most operationally reliable of the six new tools. Its limit is that the agent has to remember to use it; `sphinx-doc__sphinx-9229` remained unresolved without any `outline_file` calls, suggesting that prompt-level guidance to outline-before-reading would extend the tool's reach.

str_replace: Safe exact-string replacement

Motivation The intermediate run regressed against the baseline milestone (19 vs. 21 resolved instances) despite adding more tools, and post-hoc inspection traced the

regression to an over-broad `write_file`: agents were rewriting entire files for narrow fixes, supplying the full content from memory, and introducing transcription errors. The agent behavior suggested that a partial-edit interface was the right shape for these tasks. `str_replace` was extracted as that primitive, paired with a uniqueness invariant that `sed -i` (the obvious bash alternative) does not provide.

Interface

```
str_replace(  
    path:          str,  
    old_string:    str,          # exact substring to find  
    new_string:    str = "",     # may be empty  
    old_string_b64: str | None = None, # base64 alternatives  
    new_string_b64: str | None = None,  
)
```

The plain and base64 variants of each field are mutually exclusive; passing both returns an error from the shared `_resolve()` helper.

Implementation The tool reads the file (Docker: `cat -- <quoted-path>` via `env.execute()`; local: `Path.read_text()`), counts occurrences of `old_string`, and refuses to write unless the count is exactly 1. Zero matches return `old_string` not found in `<path>`; multiple matches return `old_string` appears N times in `<path>`; must be unique. On a unique match, the replacement is performed in Python (`str.replace(old_string, new_string, 1)`), then written back. In Docker mode, the new content is base64-encoded in Python and decoded inside the container via a `base64 -d > <path>` here-document with a `_CONTENT_` delimiter. This is the same scheme `write_file` uses, and was chosen because the base64 alphabet never contains underscores, so the delimiter has no collision-risk.

The deliberate cost is correctness over convenience: the uniqueness check forces the agent to supply enough surrounding context to disambiguate the match site. When the agent's expected-file-state diverges from reality (e.g., after intervening edits), the call fails fast with a `returncode-1` message rather than silently editing the wrong location, which is the failure mode `sed -i` produces. The `*_b64` variants exist because a single layer of JSON escaping turns out to be insufficient for content containing newlines plus quotes plus backslashes, a combination that arises in nearly every multi-line code edit.

Results `str_replace` was the most-used edit tool, invoked across 31 of the 40 final-run instances. The clearest demonstration is `sympy__sympy-16792` (Resolved, 113 steps, 14 `str_replace` calls), where the agent built a multi-block patch through repeated narrow edits to `sympy/utilities/codegen.py`. The first attempt used the plain `old_string` field with embedded newlines and failed at the JSON layer; the agent retried with `old_string_b64` and the same content, and the call succeeded. This pattern recurs across the longer-trajectory successes.

Mid-trajectory in the same run, the uniqueness invariant caught a mis-tracked file state. After an earlier insertion, the agent attempted a `str_replace` whose `old_string` referenced the pre-insertion version of a block; the call returned `old_string` not found, and the agent had to re-orient to the file's current contents.

The fast-fail on stale state is the safety net the motivation called for. A `sed -i` invocation with the same intent would have silently replaced an unintended occurrence elsewhere in the file, or no occurrence at all, with no signal to the agent.

A complementary case is `django__django-15103` (Unresolved in baseline, Resolved in final, 29 steps, 6 `str_replace` calls), where the final patch remained small while being assembled through several targeted substitutions. The tool's contribution to the final run's resolution-rate gain (21/37 vs. baseline 15/36) is large but distributed: it was rarely the sole reason a case resolved, but most of the resolved cases used it.

`insert_lines`: Insert content at specific lines

Motivation A common edit pattern in the trajectories is adding code rather than replacing it: a new method on a class, a new import, a hook before a return. `str_replace` can express insertions only by quoting one line of surrounding context as `old_string` and re-emitting it concatenated with the new content as `new_string`, every insertion thus carries an unrelated anchor line. That adds overhead in token cost and in failure modes. A dedicated `insert_lines` tool provides a first-class facility for pure-insertion edits: the agent supplies a line number and content, and the tool splices.

Interface

```
# before/after are 1-indexed and mutually exclusive
insert_lines(
    path:      str,
    content:    str,
    content_b64: str | None = None,
    before:     int | None = None,
    after:      int | None = None,
)
```

Implementation The tool validates that exactly one of `before` / `after` is supplied (both present or both absent returns returncode 2). In Docker mode, the file is split via `head -n <count>` into a `mktemp`-allocated temp file, the base64-decoded content is appended via a `base64 -d >> <tmp> here-document`, and the remainder is appended via `tail -n +<line>`; the temp file then replaces the original via `mv`. Each intermediate failure path explicitly cleans up the temp file with `rm -f`. Locally, the implementation reads the file into memory, splices the content list at the appropriate index, and writes back. Line numbers are 1-indexed externally; the `after` case

becomes index after (insert between lines), and before becomes index before - 1.

The structural tradeoff is line-number fragility. Inserting at a fixed line number is correct only relative to the file's current state; once a previous edit has shifted offsets, the same line number means a different anchor. The tool itself has no defense against this, and the burden falls on the agent to re-read or re-outline before each successive insertion. The base64 path exists for the same reason as in `str_replace`: a single layer of JSON escaping is insufficient for code blocks containing newlines, quotes, and backslashes simultaneously.

Results Two extreme cases illustrate the tool's high-variance behavior. The success case is `django__django-11880` (Resolved, 57 steps, 9 `insert_lines` calls): the agent added `import copy`, a `deepcopy` line in the field constructor, and two new test methods, recovering cleanly when the first plain-content call returned `Invalid tool payload: Expecting ',' delimiter by switching to content_b64`. The plain → base64 retry pattern observed under `str_replace` recurs here for the same reason, and the recovery is fast (typically one step).

The failure case is `sympy__sympy-13615` (Empty patch, 250 steps, 108 `insert_lines` calls). The agent tried to insert a `FiniteSet` complement clause after: 1000 in `sympy/sets/sets.py`, hit a JSON-escape error, switched to `content_b64`, then noticed the resulting file had a `SyntaxError`. From that point the agent re-attempted the insertion at the same after: 1000 over and over, but every prior insertion had pushed line 1000's original anchor downward, so each new call was inserting at a different (and increasingly wrong) position in the file. The trajectory shows the same base64 payload submitted dozens of times across the run; no patch was ever produced because the file accumulated stacked, syntactically broken duplicates of the candidate clause. `sympy__sympy-14248` exhibits the same pathology with 52 `insert_lines` calls and no patch.

Two design lessons follow. First, `insert_lines` should report the file's new line count after each successful call, so the agent observes the offset shift in the tool output rather than having to re-read the file.

Second, an idempotent form would be valuable: an insertion keyed to a one-line context anchor rather than an absolute line number, fusing `insert_lines`'s pure-insertion semantics with `str_replace`'s uniqueness invariant. The line-number addressing scheme is the wrong primitive for an agent that has no persistent model of the file between calls.

Learnings

1. Process: Iterate earlier, against real instances

The original plan for the project called for trying to deliver all tool implementations by the baseline milestone in order to validate them against a full `verified-40` run.

In practice, tools landed late and were validated primarily through unit tests. Two costs followed.

`find_definition` was projected to be the second-highest-impact tool but never returned a non-empty result across 17 invocations in the final run. A single early call against a real `verified-40` instance would have surfaced the failure early. The design of its unit tests masked it for the duration of the project.

The intermediate run’s regression against the baseline milestone (19 vs. 21 resolved) followed the same shape. The fix, narrowing `write_file` and extracting `str_replace` and `insert_lines`, was only available after observing real-trajectory failures. Earlier exposure would have shortened the corrective loop.

2. More tools and more capable tools are not strictly better

In the intermediate run, a single over-broad `write_file` invited 2,000-line whole-file rewrites, introduced transcription errors a more constrained tool would not have permitted. Resolution rate did not recover until `write_file` was narrowed and dedicated `str_replace` and `insert_lines` tools were added with strict invariants.

Each new tool encodes a hypothesis about an agent failure mode. When the hypothesis is wrong, or when the tool is permissive enough to enable a new failure mode, adding it can regress performance.

3. Constraint-bearing tools outperform permissive ones

The strongest tools in the final run were those that constrained the agent. `str_replace`’s uniqueness invariant catches stale agent state and forces re-orientation rather than silently editing the wrong site, as in `sympy__sympy-16792`. Contrast that with `sed -i`, which would have silently replaced the wrong occurrence with no signal back to the agent. Similarly, the `read_file` truncation cap with a trailing notice makes full-file dumps more visible, discouraging the “circular reads” failure mode without prohibiting full reads outright.

The contrasting design choice is `insert_lines`’s absolute-line-number addressing scheme. `sympy__sympy-13615` (Empty patch, 108 `insert_lines` calls all targeting `after: 1000`) shows the failure mode at the limit: each successful insertion shifts the anchor for the next, but the agent has no persistent model of the file between calls, so the same line number resolves to a different position every time. Line-number addressing is the wrong primitive for a coding agent. For a stateless agent, content-anchored addressing (the same uniqueness invariant `str_replace` uses) is the right one.

4. Tools shape an agent’s strategy even when they fail

The `run_tests` tool mostly did not work in the final run: Django’s `unittest`-based suite returned `ImproperlyConfigured`, and the SWE-bench harness’s 60-second container timeout overrode the tool’s own `timeout` argument. But its presence changed agent behavior. The instance `django__django-10880` resolved in the final run after six failed `run_tests` attempts pushed the agent toward Django’s own test runner, producing a 22-character patch versus the baseline’s 39-character one.

5. Tools have to be designed as a suite. How do they compose?

Refining `read_file` with `start_line / end_line`, `head / tail`, and `with_line_numbers` parameters cut token waste sharply, but did not address agent confusion. Instance `django__django-14725` (Unresolved, 31 `read_file` calls) shows the “doesn’t know where to look” pattern persisting even with slicing available.

A reader without companion localizers invites confusion: the agent will read until it runs out of budget. As a *localizer*, a tool like `outline_file`, `search_in_file`, and (had it worked) `find_definition` is more helpful to the agent.

6. Tool dispatch must verify capability inside the sandbox

Two failures share the same root cause. The `find_definition` tool runs a host-side `shutil.which("rg")` check, but in actual usage it issued the resulting command into a container that lacked `ripgrep`. Similarly, `run_tests` accepts a `timeout` argument that Python observes, but cannot exceed the harness-imposed container limit (here, 60 seconds).

In both cases, the tools were designed assuming that properties of the host Python process would hold in whatever sandbox the command would actually run in. Capability and configuration probes need to go through `env.execute()`, not through `shutil` or local subprocesses. In hindsight, this was explicitly noted in our repo in `cs427_tools_extension.md`, which we discovered belatedly, took note of, then promptly lost track of in the shuffle. *Mea culpa*.

Important Tips

Use the Environment When Possible: Tools that operate on the SWEBench dataset must issue commands inside the sandboxed environment. When the agent passes `env`, call `env.execute("...")` to run commands inside the container (working directory `/testbed`). For example, a write tool might execute:

```
env.execute("printf %s ... | base64 -d > /testbed/path")
```

Evaluation: Open GitHub Issues

Experimental Setup

The end-to-end pipeline runs `scripts/run_github_issue <ISSUE-SLUG>` once per target issue, automating both the patch-generation and validation stage, both containerized.

Several issues were attempted across multiple iterations as configuration and tooling improvements landed; the iteration discussed in each issue writeup is the final one.

Each issue is evaluated against its own acceptance criteria via a command provided to `scripts/verify_patch`, a `verify_fix.sh` script (when one is authored for the issue), or in some cases manual review.

- **Model:** vertex_ai/gemini-2.5-flash
- **Temperature:** temperature=0.0
- **Max steps:** step_limit=100
- **Token limits:** cost_limit=\$3.00 per instance
- **Number of issues evaluated:** 16
- **Pipeline command:** scripts/run_github_issue <ISSUE-SLUG>
- **Trajectory inspection:** scripts/view_trajectory
- **Results reporting:** scripts/extract_metrics <ISSUES_DIR>

Hardware

Each instance runs in an isolated container (image configurable, by default python-3.11) with a 120-second per-command timeout and a 2-hour container lifetime.

Artifact Locations

All artifacts live in subdirectories of `./<project-root>/open_github_issues/`:

- Generation traces (compressed): OWNER_REPO_NUMBER/traj.json.gz
- Patch (if generated): OWNER_REPO_NUMBER/fix.patch
- Verification scripts and associated artifacts: OWNER_REPO_NUMBER/scripts/

Performance

Metric	verified-40	GitHub issues
Resolved instances (✓)	21	9
Unresolved instances (×)	16	4
Empty-Patch instances (–)	2	3
Error instances (△)	1	0
Resolution rate (Resolved / Completed)	56.8%	69.2%

#	Slug	Result	Steps
1	zhewang2001/Project#1	–	100
2	zhewang2001/Project#2	✓	80
3	zhewang2001/Project#3	✓	10
4	zhewang2001/Project#4	✓	47
5	zhewang2001/jsoup#1	×	23
6	zhewang2001/cs427-mp2#1	–	100
7	zhewang2001/cs427-mp2#2	✓	67
8	zhewang2001/cs427-mp4#1	✓	16
9	moby/swarmkit#3196	×	27
10	containerd/fifo#56	✓	20
11	LibVNC/libvncserver#615	×	13

#	Slug	Result	Steps
12	alrevuelta/cONNXr#102	✓	24
13	fortra/impacket#1902	✓	25
14	swftools/swftools#109	✓	10
15	Lekensteyn/dmg2img#10	×	34
16	jameswalmsley/bitthunder#57	–	4

Design

1. zhewang2001/Project#1

Upstream Issue: <https://github.com/zhewang2001/Project/issues/1>

Approach & Interventions This run became an infrastructure discovery pass as much as an issue-resolution attempt. Early iterations exposed two recurring failure modes: unsafe inline edits to Gradle configuration files and environment confusion around Java and Android tooling. To address the editing failures, a `str_replace` tool was introduced so the agent could make targeted file edits without relying on brittle `sed -i` commands. The system prompt was also adjusted to steer the agent toward actually using the provided tools, since the default GitHub issue agent configuration appeared to bias the agent away from tool use.

A custom agent configuration was also added for the project. The container image was changed to an Android SDK image, `ghcr.io/cirruslabs/android-sdk:34`, so the agent would start in an environment closer to what the project required. This reduced the amount of time spent trying to diagnose missing Java or Android dependencies.

Results The issue remained unresolved. The mitigations were useful, but they solved infrastructure problems rather than the target issue itself. The safer edit tool eliminated Gradle-file mangling, and the Android SDK image reduced environment thrashing. However, the agent still did not produce a usable patch within the step and cost budget.

The main lesson from this run was that the default setup was not adequate for Android/Gradle issues. A better approach is to begin with the safer editing tool, a project-appropriate container image, and a strict time box rather than allowing the agent to spend many steps discovering those constraints from scratch.

2. zhewang2001/Project#2

Upstream Issue: <https://github.com/zhewang2001/Project/issues/2>

Approach & Interventions The run reused the Java/Android configuration lessons from the previous project issue. The default configuration had immediately encoun-

tered an unset `JAVA_HOME`, so the custom Java configuration was ported into this run.

A custom verification script was added because the issue itself had explicit acceptance criteria. The script encoded those criteria directly, giving the agent a concrete feedback loop rather than relying on vague build success or manual inspection.

Results The issue was resolved in 80 steps. The agent used the custom configuration and verification script to iteratively satisfy the acceptance criteria. The resulting patch initially did not apply cleanly because of a one-character EOF-newline problem, but that was manually corrected. After that correction, the patch applied cleanly and passed the verification script.

This approach worked because the issue was well specified and the verifier translated the acceptance criteria into executable checks. The remaining weakness was not the agent’s conceptual solution but the patch-production path, which remained sensitive to small formatting defects.

3. zhewang2001/Project#3

Upstream Issue: <https://github.com/zhewang2001/Project/issues/3>

Approach & Interventions This run adopted the tooling and configuration improvements from the earlier zhewang2001/Project attempts, including the safer file-editing tool and the project-specific Java/Android setup. The agent used `str_replace` early in the run, validating that the tool was available and useful.

Results The issue was resolved quickly, in 10 steps. The resulting patch added the `androidx.annotation` dependency and included a backup copy of the edited Gradle configuration.

The issue itself was relatively small: by the criteria specified in the issue, no source files needed annotation. The approach worked because the prior environment and editing mitigations removed most of the friction, leaving the agent with a narrow dependency-update task.

4. zhewang2001/Project#4

Upstream Issue: <https://github.com/zhewang2001/Project/issues/4>

Approach & Interventions The run began with the baseline improvements made to the default configuration. A patch was produced on the first attempt and looked plausible under manual inspection, but additional evaluation logic was then added to check whether the agent’s claimed resolution actually satisfied the issue.

Several problems emerged during evaluation. The original fix introduced a syntax error because the safer `str_replace` tool had not yet been ported into this path.

Backup files also remained in the tree and confused a verifier that checked for hard-coded strings. Build and Gradle directories were retained as well, which created corrupted patches.

The mitigations were to port the `str_replace` tool, add a `verify_fix.sh` script that delegated to a hard-coded-string linter check, and adjust the patch-generation process so `git add` excluded backup, build, and Gradle artifacts.

Results The issue was resolved after 4 iterations, with the successful iteration taking 47 steps. The successful configuration combined safer editing, executable verification, and cleaner patch collection.

The run shows that visual inspection of an agent-produced patch is not sufficient. The first patch looked valid but failed once verification was formalized. The effective improvement was to turn the issue's expected behavior into a repeatable check and to prevent non-source artifacts from entering the submitted patch.

5. zhewang2001/jsoup#1

Upstream Issue: <https://github.com/zhewang2001/jsoup/issues/1>

Approach & Interventions Three iterations were attempted. The first reached the maximum step budget. For the second iteration, a custom configuration and a verification script were added to encode the issue acceptance criteria. That helped the agent reach a substantively successful solution, but the submitted patch included build artifacts. The third iteration excluded binary output directories to avoid that source of patch conflict.

Results The issue remained unresolved in the final evaluation, even though the acceptance criteria were reportedly satisfied. The second and third iterations both reached a successful fix in substance, but neither produced a cleanly applicable patch: one was polluted by build artifacts, and the other had problematic whitespace changes.

The main failure was in patch hygiene rather than issue reasoning. A better setup would further constrain patch collection, prevent generated artifacts and broad formatting changes from entering the diff, and possibly include a pre-submission `git diff --check` / clean-apply validation step before the agent submits.

6. zhewang2001/cs427-mp2#1

Upstream Issue: <https://github.com/zhewang2001/cs427-mp2/issues/1>

Approach & Interventions Two iterations were attempted. The first iteration churned on configuration-file edits before sufficiently orienting itself in the project. For the second iteration, a custom Maven-oriented configuration was used. The

prompt explicitly nudged the agent toward using an autoformatter rather than manually correcting linter violations one by one.

An acceptance-criteria verification script was also used to provide a more concrete target.

Results No patch was produced. The second iteration did make progress: it successfully reached and ran the autoformatter around steps 14–17. After that, it churned on a build failure caused by missing dependencies and then got pulled into manual linter-fix work. The run also encountered environment issues, including a missing container, and was time-boxed because API rate limits were being saturated.

The likely improvement would be to make the Maven environment fully reproducible before launching the agent, including dependency availability and the exact formatter/linter commands. The prompt-level instruction to prefer autoformatting helped, but it was not enough to overcome environment instability.

7. zhewang2001/cs427-mp2#2

Upstream Issue: <https://github.com/zhewang2001/cs427-mp2/issues/2>

Approach & Interventions This run pre-emptively incorporated the improvements from previous zhewang2001 issue attempts. A custom Maven configuration was used, including an explicit tip on how to upgrade a Maven project to JDK 17. A verification script encoded the acceptance criteria as comments and used `mvn help:evaluate` plus `mvn -B -ntp clean install` to validate the expected project state.

The run also exposed difficulty with JSON-encoded tool input, which informed a later improvement to writer tools.

Results The issue was resolved. Despite stumbling on tricky JSON-encoded input, the agent recovered, produced a patch, and the patch applied cleanly while satisfying the acceptance criteria.

This worked because the run started with accumulated infrastructure improvements rather than rediscovering them. The verification script gave the agent a precise target, while the custom Maven/JDK guidance reduced unnecessary exploration.

8. zhewang2001/cs427-mp4#1

Upstream Issue: <https://github.com/zhewang2001/cs427-mp4/issues/1>

Approach & Interventions This run began from the mature configuration state produced by earlier GitHub issue and `verified-40` work. No additional run-specific mitigation was required. The agent had access to the current tool implementations and the lessons from previous issue attempts.

The acceptance criteria were explicitly encoded, effectively serving as a TODO list for the agent. This helped keep the run focused.

Results The issue was resolved in 16 steps. The agent correctly recognized that specific test failures were expected because the implementation under test was still a stub. The task was to refactor tests into parameterized form, not to implement the underlying SBFL logic.

The approach worked because the acceptance criteria were clear enough to distinguish structural test refactoring from unrelated implementation failures. The agent verified that all five original test cases were represented as parameterized inputs, that duplicated test logic had been consolidated, and that the parameterized tests compiled and ran via Maven even though the stub implementation still caused assertion failures.

9. moby/swarmkit#3196

Upstream Issue: <https://github.com/moby/swarmkit/issues/3196>

Approach & Interventions Two iterations were attempted. The first used the default baseline and wasted several steps trying to install Go because the default `python:3.11` container did not include the Go toolchain. The second iteration switched to a `golang:1.24` image, added issue-specific guidance pointing the agent toward `peer.go` and `peer.stop()`, and added a `verify_fix.sh` script that injected a `golink`-based mirror of the reporter's `TestSendRemoved` reproduction.

Results The issue was marked unresolved by evaluation, but the trajectory indicates the agent likely found the correct fix. The unresolved result appears to have come from flaky verification rather than an incorrect fix. The verifier relied on `TestSendRemoved`, which was already intermittently unstable in the environment due to transient gRPC teardown failures. A standalone leak test would likely have produced a more reliable signal.

A secondary environment lesson was that even the `golang:1.24` image could report `go: not found` because `bash -lc` sourced `/etc/profile`, which overwrote the Docker image's `PATH`. The agent had to explicitly run `export PATH=$PATH:/usr/local/go/bin`.

10. containerd/fifo#56

Upstream Issue: <https://github.com/containerd/fifo/issues/56>

Approach & Interventions Two iterations were attempted. The issue concerned a goroutine leak in `OpenFifo(context.Background(), ..., O_RDONLY|O_CREAT|O_NONBLOCK, 0600)`. In the faulty behavior, one goroutine in `openFifo` blocked forever on `<-ctx.Done()` because the context was

`Background()`, while the nonblocking read-only open never completed and the relevant opened/closed signals never fired.

The first iteration was aborted after extensive edit-tool thrashing. The agent spent more than 50 steps fighting JSON escaping in tools such as `str_replace`, `replace_lines`, and `insert_lines`, broke the build, and then recovered line by line. The second iteration used a `golang:1.24` image, explicit PATH guidance requiring `export PATH=$PATH:/usr/local/go/bin`, and heredoc-only edit guidance. The verification script dropped in the reporter's `TestFifoNocancel` test directly from the issue body.

Results The issue was resolved in the second iteration in about 20 steps. The verification test went from failing before the fix to passing after the fix. The patch added `case <-f.closing:` to the first goroutine's select and added a `defer f.Close()` on the `openFifo` error path, allowing the closing signal to propagate and wake blocked goroutines.

The run also revealed a caveat: the agent's whole-file rewrite dropped the upstream copyright header, the `//go:build !windows` constraint, and the `OpenFifoDup2` function. These were unrelated regressions and would need to be restored before an upstream-quality patch could be submitted.

The main process lesson is that heredoc-only guidance dramatically reduced tool friction. Combined with a self-contained reproduction test, it turned a 91-step failed attempt into a roughly 20-step successful one.

11. LibVNC/libvncserver#615

Upstream Issue: <https://github.com/LibVNC/libvncserver/issues/615>

Approach & Interventions Two iterations were attempted. The first iteration identified the bug but ended with an `Empty_Patch` because the agent ran `cmake .` inside the repository root. That polluted the tree and broke patch collection. It also fought JSON escaping in `replace_lines` and broke an `#endif`.

The second iteration used a `gcc:13` image, explicit out-of-source build guidance using `/tmp/build`, heredoc-oriented edit guidance, and a `verify_fix.sh` script that built `vncserver` out of source and statically checked that `listenerRun` referenced `FD_SETSIZE`.

Results The issue remained unresolved. During the run, `verify_fix.sh` passed: the build completed and the static check confirmed that `FD_SETSIZE` bounds logic had been added. However, final evaluation failed for two reasons.

First, the generated patch was malformed and `git apply` rejected it as corrupt. Second, the patch had a regression: an edit removed the `FD_ZERO(&listen_fds);` line immediately before the first `FD_SET` block. Without `FD_ZERO`, stale bits could remain across loop iterations.

The conceptual fix was correct: add bounds checks around the `FD_SET` calls, include `<sys/select.h>`, and log overflow conditions. The implementation path failed because the edit tooling damaged adjacent code and the static verifier was too shallow to catch the `FD_ZERO` regression. A stronger runtime reproducer that opened more than 1024 file descriptors under ASAN would have been better, but likely too expensive for the iteration budget.

12. alrevuelta/cONNXr#102

Upstream Issue: <https://github.com/alrevuelta/cONNXr/issues/102>

Approach & Interventions No special mitigation was needed. The first run was interrupted because of API limits and an execution-loop issue. The agent was rerun, and the final patch was manually extracted after confirming the fix.

The target problem involved missing null checks for `searchAttributeNyName` in `Constant-12` and `MaxPool-12` handling.

Results The issue was resolved. The agent added the missing null checks in both relevant locations. The earlier failure was environmental rather than conceptual: API limits interrupted the first attempt, but the rerun produced a valid fix.

13. fortra/impacket#1902

Upstream Issue: <https://github.com/fortra/impacket/issues/1902>

Approach & Interventions Seven iterations were attempted. Early patches did not apply cleanly, and naive verification by running `pytest` encountered environment and application-level problems, including missing `pytest` and missing project dependencies.

To stabilize the run, a custom configuration with tailored instructions was added, along with a `verify_fix.sh` containing project-specific test commands.

Results The issue was resolved. The successful setup depended on replacing broad, naive `pytest` execution with a project-aware verification script and agent instructions adapted to the repository. Once verification reflected the project's actual test requirements, the agent could produce a cleanly applicable resolving patch.

14. swftools/swftools#109

Upstream Issue: <https://github.com/swftools/swftools/issues/109>

Approach & Interventions No special mitigation was needed. The issue was handled in one iteration and was reported as resolved without custom configuration or tooling.

Results The issue was resolved. The patch applied cleanly and was verified manually.

15. Lekensteyn/dmg2img#10

Upstream Issue: <https://github.com/Lekensteyn/dmg2img/issues/10>

Approach & Interventions A simple verification script was added to rebuild `dmg2img` and rerun the issue reproduction input. The issue description provided both the reproduction command and the crashing artifact, and the project README described custom build requirements.

During the trajectory, the agent downloaded `heap-overflow-adc-66.zip`, extracted it, and ran the reproduction command:

```
./dmg2img -i ./heap-overflow-adc-66 -o /dev/null
```

A clang container image was used during evaluation based on the project README's build guidance, but it was not made available to the agent itself.

Results The issue remained unresolved. In the agent trajectory, the patched program no longer segfaulted when the reproduction command was run, so the agent concluded that the issue was fixed. During later evaluation, the project built successfully, but the same reproduction still segfaulted.

The discrepancy suggests that the agent's local verification and the final evaluation environment did not test the same effective state, or that the fix only masked the crash under the agent's conditions. More investigation would be needed to compare inputs, in particular the choice of compiler.

16. jameswalmsley/bitthunder#57

Upstream Issue: <https://github.com/jameswalmsley/bitthunder/issues/57>

Approach & Interventions No mitigation was needed because the issue had already been resolved upstream. The target complaint was that `bt_system_init` did not return a value of type `BT_ERROR`.

Results The run produced `Empty_Patch`. The current repository HEAD already had the fix: `bt_system_init` in `os/src/bt_main.c` returned `Error` of type `BT_ERROR`. The agent correctly identified that no changes were necessary.

This was a stale issue rather than a failed repair attempt, suggesting the utility of a preflight check for already-resolved issues before spending agent budget on them.

Learnings

1. Start the agent in a project-appropriate environment

The default `python:3.11` container is rarely a useful starting point for a non-Python project. `moby/swarmkit#3196` wasted several steps in its first iteration trying to install Go before switching to `golang:1.24`; the default configuration on `zhewang2001/Project#2` immediately hit an unset `JAVA_HOME` and required porting in a custom Java/Android setup; `LibVNC/libvncserver#615` switched to `gcc:13` for its second iteration; `zhewang2001/Project#1` substituted `ghcr.io/cirruslabs/android-sdk:34` so the agent would not spend its budget rediscovering missing Android tooling. The agent that opens onto a project-appropriate image spends its budget on the issue rather than on environmental archaeology.

Two orthogonal observations reinforce the same conclusion. First, even an “appropriate” image can hide footguns: `golang:1.24` reported `go: not found` because `bash -lc sourced /etc/profile`, which overwrote the image’s `PATH` (`swarmkit#3196`, `containerd/fifo#56`). The corrective is a one-line `export PATH=$PATH:/usr/local/go/bin` injected into the agent’s preamble. Second, the default GitHub-issue agent configuration itself biased the agent away from tool use; `Project#1` had to adjust the system prompt to steer the agent toward actually invoking the tools it was given.

2. Encode acceptance criteria as a todo list and executable verifier

The single largest behavioral change observed in this evaluation came from giving the agent a `verify_fix.sh` that encoded the issue’s acceptance criteria as executable checks. `zhewang2001/Project#2` resolved in 80 steps with such a script; `cs427-mp2#2` and `cs427-mp4#1` resolved cleanly with criteria-as-comments verifiers; `impacket#1902` only resolved after seven iterations because the early attempts used a generic `pytest` invocation that surfaced environment failures rather than the actual project test signal.

Issue `containerd/fifo#56` is the clearest illustration of the magnitude: lifting the reporter’s `TestFifoNoCancel` test directly from the issue body into a verification script turned a 91-step failed attempt into a roughly 20-step successful one. The verification script does two things at once. It tells the agent when to stop, replacing vague “looks right” judgments with a binary signal. It constrains the search space. The agent edits toward the verifier rather than wandering through plausible-looking changes. Without it, the most common failure mode is thrashing on superficially related symptoms.

3. Hint at known pitfalls, file locations, and reproduction steps

Issue-specific guidance reduced step counts substantially across the runs that used it. `swarmkit#3196` pointed the agent at `peer.go` and `peer.stop()`. `LibVNC/libvncserver#615` injected explicit out-of-source-build guidance (`/tmp/build`) after the first iteration’s `cmake .` polluted the tree. `cs427-mp2#2`

shipped a tip on upgrading Maven projects to JDK 17. `containerd/fifo#56` switched to heredoc-only edit guidance after the first iteration spent more than 50 steps fighting JSON escaping in the editing tools.

The unifying observation is that the issue body and the project README usually contain enough information to materially reduce the agent’s exploration budget, but that information must be hoisted into the agent’s starting prompt rather than left for the agent to rediscover. Each hint costs minutes to add and saves dozens of steps.

4. Patch hygiene is crucial

A surprising number of runs reached a substantively correct fix and still failed evaluation due to patch-production problems. `jsoup#1` produced a working fix on two iterations, neither of which yielded a cleanly applicable patch (build artifacts on one, problematic whitespace on the other). `LibVNC/libvncserver#615` had the right conceptual fix but the generated patch was malformed and `git apply` rejected it as corrupt. `Project#2` tripped on a single missing EOF newline. `Project#4` had to add `git add` exclusions for backup, build, and Gradle artifacts before its patches stopped corrupting evaluation.

Two corollaries follow. First, agent-side `verify_fix.sh` passing is necessary but not sufficient: a pre-submission `git diff --check` and clean-apply validation belongs in the pipeline. Second, the agent’s verifier must be a faithful surrogate for the evaluation environment. `Lekensteyn/dmg2img#10` ran the reproduction inside the agent’s environment and observed no segfault; the same reproduction still segfaulted under evaluation, almost certainly because the agent did not have access to the `clang` container the evaluator used. Verifier-eval divergence silently flips a “resolved” trajectory into an unresolved result. A shallow verifier creates the same risk: `LibVNC/libvncserver#615`’s static check confirmed `FD_SETSIZE` bounds were added but missed an unrelated regression in which `FD_ZERO(&listen_fds)` had been deleted.

5. Improvements compound across runs

The strongest single predictor of success after the first few issues was whether the run began with the accumulated infrastructure from prior issues. `Project#3` resolved in 10 steps because the safer file-editing tool, the Java/Android container, and the prompt adjustments from `Project#1` and `Project#2` were already in place. `cs427-mp2#2` and `cs427-mp4#1` resolved with little or no run-specific intervention for the same reason. The mature configuration is the cumulative artifact of every earlier failure, and porting it forward eliminates the rediscovery tax. The corresponding warning is that improvements which fail to port forward reproduce the same failures: `Project#4`’s first patch introduced a syntax error precisely because the `str_replace` tool from earlier runs had not yet been wired into that path.

6. A cheap preflight saves budget

Issue `bitthunder#57` produced an `Empty_Patch` because the issue had already been resolved upstream. The agent correctly identified that no change was necessary, but four steps and the surrounding tooling overhead were spent confirming it. A

trivial preflight that runs the issue's reproduction or checks the relevant code state before launching the agent would have skipped the run entirely. The same shape of check could rule out other cheap-to-detect cases (missing repro artifact, build broken at HEAD, dependencies unavailable) before any agent steps are spent.

Comparison with SWE-Bench instance runs

The GitHub-issues run resolved 9 of 13 completed issues (69.2%); in the `verified-40` run, 21 of 37 (56.8%). The gap is largely explained by the per-issue investment put into the former, which `verified-40` cannot use by construction.

Dimension	<code>verified-40</code>	GitHub issues
Evaluation Attempts	Fixed benchmark subset One attempt per instance	Hand-selected real issues Multiple iterations allowed for some issues
Environment	Shared SWE-bench image family	Project-specific containers when needed
Validation	Hidden benchmark tests	Issue-specific verifier, command, or manual review
Agent guidance	Shared prompt and tools	Per-issue hints and reproduction guidance
Reported resolution	Benchmark score for a fixed agent	Agent plus per-issue engineering support

Why the GitHub issue setting looks better

The GitHub issue results should be read as the performance of the agent *with scaffolding*, not as a clean benchmark score. The `verified-40` run was a single-shot evaluation under one shared configuration: one container image family, one prompt, one tool set, and one attempt per instance. The GitHub issue runs admitted bespoke containers (`golang:1.24`, `gcc:13`, `android-sdk:34`), per-issue verification scripts, hand-written hints, and multiple iterations for some issues. A fixed benchmark is designed to exclude such interventions.

That scaffolding mattered because open GitHub issues often contain more usable operational context than a benchmark instance exposes to the agent. Issue bodies and repo READMEs supplied reproduction commands, acceptance criteria, stack traces, and sometimes the likely file or function. When that information was hoisted into the prompt or into `verify_fix.sh`, the task changed from “search an unfamiliar project until something looks plausible” to “satisfy this concrete reproducer or checklist.” This is why `containerd/fifo#56` could move from a long, tool-friction-heavy failure to a short successful run once the reporter’s reproduction was encoded directly.

The GitHub issue workflow also allowed cumulative engineering support. Early failures on Java, Android, Go, CMake, patch collection, and JSON escaping became configuration changes that later issues inherited. By contrast, `verified-40` intentionally asks whether a single agent configuration works broadly without per-instance repair work between attempts.

What transfers back to benchmark runs

The most transferable lessons are about tool and pipeline design. Patch hygiene failures, sandbox capability mismatches, and verifier-evaluation divergence appeared in both settings. A pre-submission `git diff --check`, clean-apply validation, sandbox-side capability probes, and content-anchored edit tools are not GitHub-issue-specific; they would make both evaluations more reliable.

The same is true for localizer tools. The GitHub issue setting shows the value of narrowing the search space before editing, whether that narrowing comes from a hand-written hint, an issue reproduction, `outline_file`, or `search_in_file`. In `verified-40`, manual hints are unavailable, so the tooling has to supply more of that localization pressure.

What the comparison cannot prove

The comparison does not prove that the agent is inherently better on open GitHub issues than on SWE-bench instances. The issue set was smaller, manually selected, and unevenly supported; several issues received custom containers, custom verifiers, or repeated attempts. The 69.2% result is therefore best interpreted as evidence that agent performance improves sharply when the surrounding workflow turns issue knowledge into executable feedback.

The benchmark result answers a different question: how far a fixed agent setup gets under controlled, repeatable conditions. The GitHub issue result answers a more practical question: how effective the same agent can be when a developer is allowed to build a small harness around each real issue. Both are useful, but they measure different parts of the system.